Goodput-Aware Adaptive Speculative Decoding for Batched LLM Serving: Midterm Report

Vaishnavi Athrey Ramesh
Carnegie Mellon University
vathreyr@andrew.cmu.edu

Aritra Raychaudhuri
Carnegie Mellon University
araychau@andrew.cmu.edu

Abstract

Speculative decoding accelerates large language model inference by letting a small drafter model propose a block of gamma tokens that the large target model verifies in a single forward pass. The achievable speedup depends on the speculation length gamma, which current serving systems such as vLLM expose as a static flag. We argue that the best gamma is load dependent and should be chosen online. This report presents three pieces of midterm progress. First, a roofline analysis that predicts how the throughput optimal gamma varies with batch size and acceptance rate, used to guide the design before implementation. Second, an adaptive controller that estimates the acceptance rate online and selects gamma by maximizing a goodput proxy. Third, a discrete event serving simulator that validates the controller against static, no speculation, and oracle baselines. In simulation the throughput optimal gamma falls from about 4 at batch size 1 to 1 at large batch, confirming that no single static gamma is best across a load sweep, and the adaptive controller tracks the per step oracle while beating every static setting on both throughput and goodput. We close with the plan to calibrate the cost model and reproduce these comparisons on real GPUs.

1 Introduction

When a language model serves requests, it decodes one token per forward pass of a large target model, and that forward pass is dominated by reading the model weights and the key value cache from memory. Speculative decoding (Leviathan et al. [1], Chen et al. [2]) reduces the number of target passes by having a cheap drafter propose gamma candidate tokens that the target then verifies in parallel. When the drafter is accurate, several tokens are emitted per target pass rather than one.

The speedup is governed by two quantities, the speculation length gamma and the per token acceptance rate alpha. Setting gamma is a trade off. A larger gamma emits more tokens per step when acceptance is high, but it also forces the target to verify batch times (gamma plus 1) token positions, which wastes compute whenever those extra tokens are rejected. The right value therefore depends on how busy the GPU is. At low load the target decode is memory bound and the GPU has idle compute, so verifying extra speculative tokens is almost free. At high load the target decode becomes compute bound and the wasted verification competes with useful work, which lowers throughput and inflates the per output token latency that serving SLOs are written against.

Despite this, production stacks fix gamma as a static configuration flag. Our idea is to treat gamma as an online scheduling decision. We build a controller that adapts gamma from the live acceptance rate and the current batch size so as to maximize goodput, meaning useful tokens per unit of GPU time, which under a latency SLO corresponds to requests per second that meet a target per output token latency. Our goal for the project is to show that this controller beats any static gamma across a load sweep, first in simulation and then on real hardware. This midterm report covers the design, the roofline analysis that motivated it, and the simulation study.

2 Related Work

Leviathan et al. [1] and Chen et al. [2] introduced speculative decoding and the rejection sampling scheme that makes the accepted output distribution identical to standard decoding. Their analysis gives the expected number of tokens emitted per step as a function of alpha and gamma, which we reuse directly in our objective. EAGLE and EAGLE-3 (Li et al. [3]) train a lightweight drafter head that conditions on the target features and reach higher acceptance than simple drafters, and we adopt EAGLE-3 as a high acceptance comparison in the final phase. vLLM (Kwon et al. [4]) is the serving system we target. It implements continuous batching and speculative decoding, but the speculation length is set once at engine construction. The roofline model (Williams et al. [5]) is the performance model we use to reason about when a decode step is memory bound versus compute bound, which is the regime distinction our whole approach rests on. Our contribution is to combine these pieces into an online, goodput aware controller for the speculation length and to study it across load, drafter, and hardware.

3 Methodology

3.1 Roofline cost model

We model one propose then verify macro step. For a batch of B requests at speculation length gamma, the drafter runs gamma sequential steps and the target verifies B times (gamma plus 1) positions in one pass. A single target pass over n positions costs

t_target(n) = t_mem + t_flop times n,

where t_mem is the memory bound floor of a decode step, dominated by reading weights and the key value cache, and t_flop is the marginal compute cost per token once the GPU is compute bound. The macro step cost is

t_macro(B, gamma) = gamma times t_draft(B) + t_target(B times (gamma + 1)).

Following Leviathan et al. [1], the expected number of accepted tokens emitted per request per step is E[tokens | alpha, gamma] = (1 minus alpha to the power (gamma plus 1)) divided by (1 minus alpha). The controller maximizes a goodput proxy, accepted tokens per unit of GPU time,

gamma* = argmax over gamma of E[tokens | alpha, gamma] divided by t_macro(B, gamma).

The key qualitative prediction follows from the shape of t_macro. Because the verification term grows with B times (gamma plus 1) times t_flop, the denominator penalizes long speculation exactly when the batch is large and the GPU is compute bound. The optimal gamma should therefore shrink as B grows and grow as alpha grows. This closed form prediction is what justified building an adaptive controller, and it is the central hypothesis we test on hardware in the final report.

3.2 Adaptive controller

The controller keeps an exponential moving average estimate of alpha from realized accept and reject outcomes, and at each step selects gamma* using the objective above. It needs only three inputs, the current running batch size, the online acceptance estimate, and the cost model, all of which a serving scheduler already has at scheduling time. One implementation detail matters. The acceptance feedback is folded in as a fraction in the interval from 0 to 1 rather than a rounded integer, because rounding collapses the estimate for gamma equal to 1 and biases the controller toward the minimum speculation length. We also implement an oracle policy that selects gamma using the true current acceptance rate, which gives an upper bound on what any acceptance aware policy can achieve.

3.3 Serving simulator

We built a discrete event simulator of a server doing speculative decoding under continuous batching. It models Poisson arrivals at rate lambda, per request output lengths so requests finish and leave the batch, a maximum batch cap with a queue when overloaded, heterogeneous per request acceptance with optional within request drift, and realistic accept sampling where gamma Bernoulli draws determine the accepted run length. All timing comes from the cost model and all policy decisions come from the controller, so the same controller object can later be moved into vLLM without changing its logic. The simulator reports throughput, mean and tail per output token latency, SLO attainment, goodput, mean batch size, and mean gamma.

3.4 Metrics

For the policy study we report throughput in tokens per second, goodput in requests per second meeting a per output token latency SLO, and per output token latency at the median, ninetieth, and ninety ninth percentiles. The acceptance rate is tracked as the fraction of drafted tokens accepted.

4 Results

All numbers in this section are outputs of the simulator with order of magnitude cost constants. Calibrating those constants against real microbenchmarks is part of the remaining work.

4.1 Load versus best speculation length

Table 1 reports the throughput optimal static gamma as a function of batch size from the cost model at a fixed acceptance rate of 0.72. The optimum falls from 4 at batch size 1 to 1 once the batch is large, which matches the roofline prediction that long speculation stops paying off once the verification pass becomes compute bound. Figure 1 shows the full tokens per second versus gamma curves for several batch sizes alongside the optimal gamma curve.

Table 1. Throughput optimal static gamma versus batch size (cost model, alpha equals 0.72).

Batch size	1	4	16	32	64	128
Best gamma	4	3	2	1	1	1

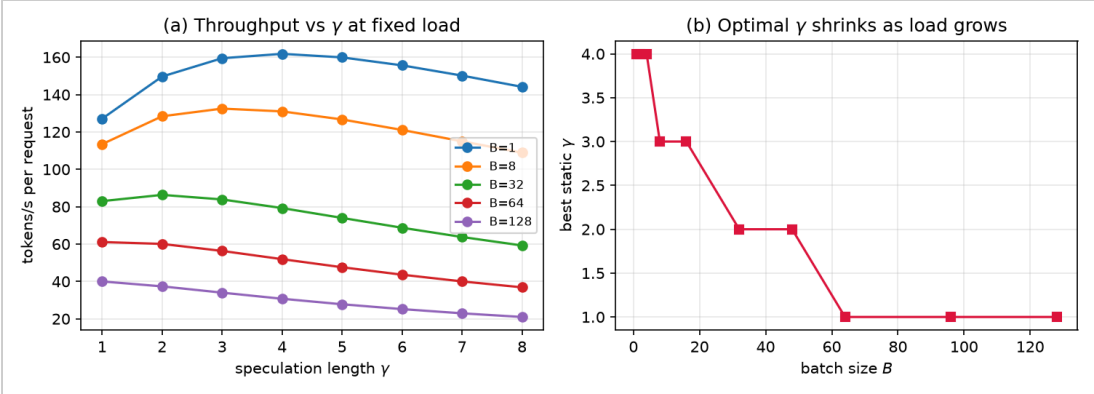


Figure 1. Tokens per second versus gamma at several batch sizes (left) and the optimal static gamma versus batch size (right). The optimum shrinks as load grows.

4.2 Policy comparison across load

Figure 2 compares no speculation, two static settings, the adaptive controller, and the oracle across an arrival rate sweep on throughput and goodput. The adaptive controller tracks the oracle and stays above every static setting across the whole sweep, because a static gamma that is tuned for low

load is wasteful at high load and a static gamma tuned for high load leaves speedup on the table at low load.

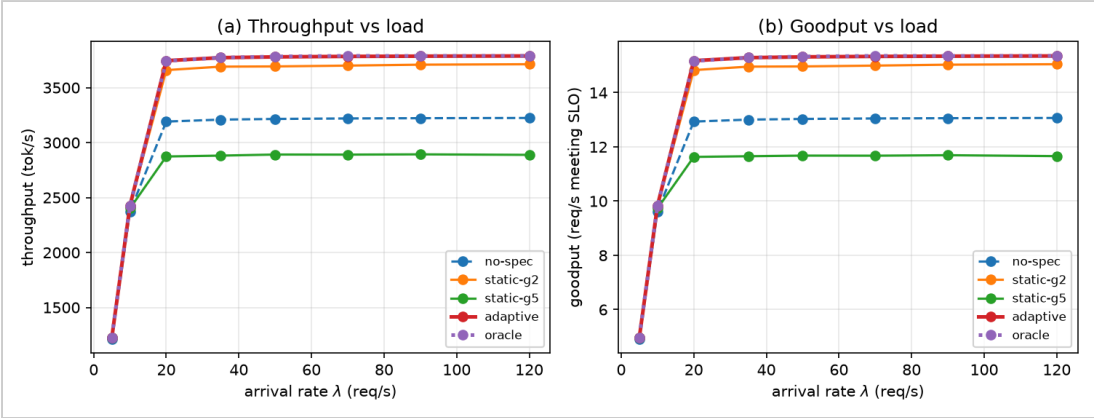


Figure 2. Throughput (left) and goodput (right) versus arrival rate for each policy. The adaptive controller and the oracle sit above the static baselines across the load range.

4.3 Within generation acceptance drift

In a workload where acceptance decays as a generation gets longer, the controller lowers gamma as the estimated acceptance drops and keeps a higher throughput and goodput than the static baselines. This confirms that the online acceptance estimate, not just the batch size, is doing useful work.

5 Discussion and Plan

The simulation establishes that the controller logic is correct and that the mechanism can help in principle. The open question is whether real hardware actually exhibits the regime the roofline assumes. If a real decode step is so memory bound that a static gamma is already close to optimal across the batch range, the benefit of adaptation will be small, and that itself is a useful finding. The plan for the final phase is as follows. First, integrate the controller with vLLM speculative decoding, reading the running batch size to choose gamma and feeding the rejection sampler accepted counts back as a fraction, reusing the exact simulator controller so the two stay consistent. Second, calibrate t_mem and t_flop from real decode microbenchmarks by sweeping batch size and measuring step latency, and replace the guessed constants. Third, run a speculation length by batch size grid and a load sweep comparing no speculation, static gamma in two four six, and the adaptive policy on throughput, goodput, and tail latency. We will use n-gram or prompt lookup speculation as a portable drafter that needs no draft model, and EAGLE-3 as a trained, higher acceptance comparison. We identified two risks early. One is the portability of vLLM speculative decoding kernels on older GPUs. The other is that the hardware may sit in a different point on the roofline than assumed, which would change how much adaptation helps.

6 Conclusion

We framed the speculation length in speculative decoding as an online scheduling decision rather than a static hyperparameter, derived from a roofline model that the throughput optimal gamma should shrink with batch size and grow with acceptance, and built an adaptive controller and a serving simulator that confirm this and show the controller beating every static gamma across a load sweep. The remaining work is to calibrate the cost model on real GPUs and to reproduce the policy comparison on vLLM with both an n-gram drafter and EAGLE-3.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In Proc. ICML, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, et al. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[3] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In Proc. ICML, 2024.

[4] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In Proc. SOSP, 2023.

[5] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4):65 to 76, 2009.

Team Contributions

Member	Responsibilities
Vaishnavi Athrey Ramesh	Roofline cost model, adaptive controller, experiments
Aritra Raychaudhuri	Serving simulator, baseline policies, evaluation and report